Machine learning Lab Observation
ITA0614

Experiment 1

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Name : Sakthi K S

Reg.No : 192412247

Problem 1 : EnjoySport

Aim

To implement the FIND-S algorithm on the EnjoySport training data stored in a .CSV file and to find the most specific hypothesis consistent with all the positive examples.

Algorithm

1. Initialise h to the most specific hypothesis in H, <0, 0, ..., 0>.
2. For each training example x:

   If x is a positive example: for each attribute constraint a in h, if a is satisfied by x do nothing; otherwise replace a in h by the next more general constraint that is satisfied by x (the first positive example fills in its own values and a conflicting value becomes ?).

   If x is a negative example: ignore it and leave h unchanged.
3. Output the hypothesis h.

Program:

find_s_enjoysport.py — Python 3

```python
# Program 1(a): FIND-S algorithm on EnjoySport data stored in a .CSV file
import csv

with open('enjoysport.csv') as f:
    rows = list(csv.reader(f))
head, data = rows[0], rows[1:]
n = len(head) - 1

def fmt(h):
    return "<" + ",".join(h) + ">"

h = ['0'] * n                                # most specific hypothesis
print("Initial h =", fmt(h)); print()

for k, row in enumerate(data, 1):
    x, label = row[:n], row[n]
    if label == 'Yes':                       # negative examples are ignored
        for i in range(n):
            if h[i] == '0':
                h[i] = x[i]
            elif h[i] != x[i]:
                h[i] = '?'
    print(f"Example {k} ({label:>3}):  h = {fmt(h)}")

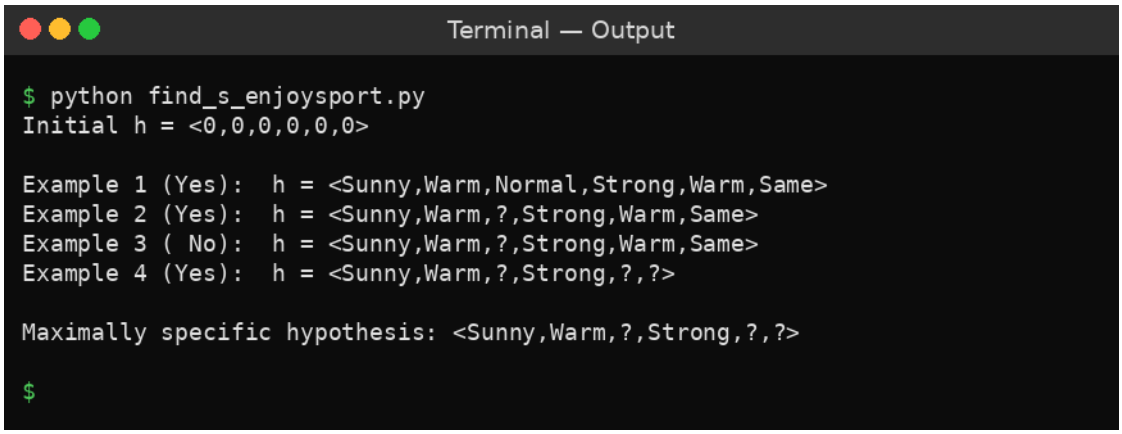
print()
print("Maximally specific hypothesis:", fmt(h))
```

Input

EnjoySport data set (enjoysport.csv): 4 examples, 6 attributes (Sky, AirTemp, Humidity, Wind, Water, Forecast) and the target concept EnjoySport (Yes / No). The attribute values are read directly from the file.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Output



Result

The maximally specific hypothesis for the EnjoySport data is <Sunny, Warm, ?, Strong, ?, ?>. The first example sets h to <Sunny, Warm, Normal, Strong, Warm, Same>, the second example generalises Humidity to ?, the negative third example is ignored, and the fourth example generalises Water and Forecast to ?. Sport is therefore enjoyed on sunny, warm days with a strong wind.

Problem 2: Loan approval

Aim

To implement the FIND-S algorithm using the pandas library on a loan approval data set and to find the most specific hypothesis that describes the approved applications.

Algorithm

1. Initialise h to the first positive training example.
2. For each remaining training example x:

   If x is a positive example: for each attribute constraint a in h, if a is satisfied by x do nothing; otherwise replace a in h by the more general constraint ? (the value is no longer fixed).

   If x is a negative example: ignore it and leave h unchanged.
3. Output the hypothesis h.

Program:

find_s_loan.py — Python 3

```python
# Program 1(b): FIND-S algorithm on loan approval data using pandas
import pandas as pd

df = pd.read_csv('loan.csv')
attrs = list(df.columns[:-1])
target = df.columns[-1]

h = None                                    # first positive example starts h
for idx, row in df.iterrows():
    if row[target] == 'Yes':
        x = list(row[attrs])
        if h is None:
            h = x[:]
        else:
            h = [a if a == b else '?' for a, b in zip(h, x)]
    print("Row", idx + 1, "->", f"{row[target]:>3}", "| h =", h)

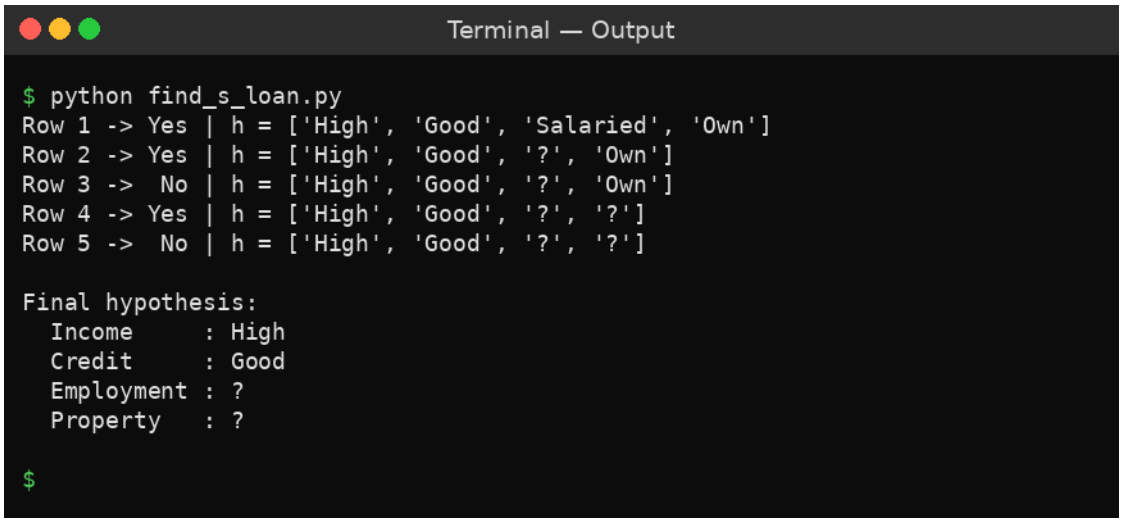
print()
print("Final hypothesis:")
for a, v in zip(attrs, h):
    print(f"  {a:<11}: {v}")
```

Input

Loan data set (loan.csv): 5 examples, 4 attributes (Income, Credit, Employment, Property) and the target concept Approved (Yes / No), loaded as a pandas DataFrame.

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Own	Yes
High	Good	Self-employed	Own	Yes
Low	Poor	Salaried	Rent	No
High	Good	Salaried	Rent	Yes
Medium	Poor	Self-employed	Rent	No

Output



Result

After processing all five rows the hypothesis is <High, Good, ?, ?>. Row 1 initialises h, Row 2 generalises Employment to ?, Row 4 generalises Property to ?, and the negative Rows 3 and 5 leave h unchanged. A loan is therefore approved whenever the income is High and the credit history is Good, whatever the employment type or property status.

Problem 3: Mushroom edibility

Aim

To implement FIND-S as a reusable function on mushroom data stored in the program as tuples and to display how the hypothesis is generalised after every positive example.

Algorithm

1. Initialise h to the first positive training example.
2. For each remaining training example x:

   If x is a positive example: for each attribute constraint a in h, if a is satisfied by x do nothing; otherwise replace a in h by the more general constraint ? (the value is no longer fixed).

   If x is a negative example: ignore it and leave h unchanged.
3. Output the hypothesis h.

Program:

find_s_mushroom.py — Python 3

```python
# Program 1(c): FIND-S on mushroom data stored as a list of tuples
data = [
    ("Convex", "Brown",  "None", "Woods", "Edible"),
    ("Convex", "Brown",  "None", "Grass", "Edible"),
    ("Flat",   "White",  "Foul", "Woods", "Poisonous"),
    ("Convex", "Yellow", "None", "Woods", "Edible"),
    ("Bell",   "Brown",  "Foul", "Grass", "Poisonous"),
]
attrs = ["Cap", "Colour", "Odour", "Habitat"]

def find_s(examples, positive):
    h = None
    for *x, label in examples:
        if label != positive:
            continue                        # skip negative examples
        h = x if h is None else [a if a == b else '?' for a, b in zip(h, x)]
        print("  after", tuple(x), "->", tuple(h))
    return h

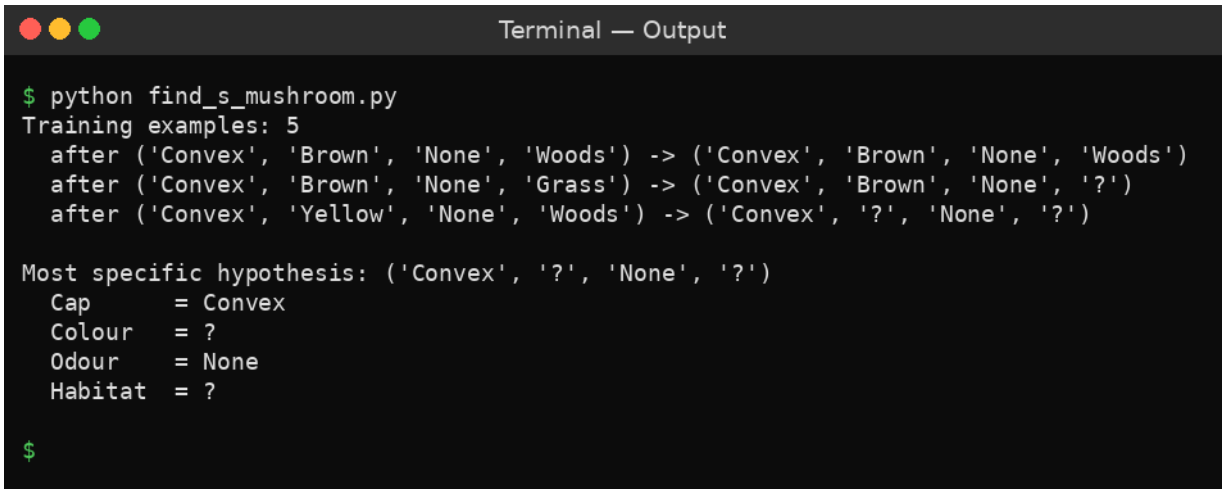
print("Training examples:", len(data))
h = find_s(data, "Edible")
print()
print("Most specific hypothesis:", tuple(h))
for a, v in zip(attrs, h):
    print(f"  {a:<8} = {v}")
```

Input

Mushroom data set: 5 examples, 4 attributes (Cap, Colour, Odour, Habitat) and the target concept Edible / Poisonous. The data is stored in the program as tuples, and Edible is treated as the positive class.

Cap	Colour	Odour	Habitat	Class
Convex	Brown	None	Woods	Edible
Convex	Brown	None	Grass	Edible
Flat	White	Foul	Woods	Poisonous
Convex	Yellow	None	Woods	Edible
Bell	Brown	Foul	Grass	Poisonous

Output



Result

The most specific hypothesis is <Convex, ?, None, ?>. Habitat is generalised to ? after the second edible example and Colour is generalised to ? after the fourth, while the two poisonous examples are skipped. In this data an edible mushroom has a convex cap and no odour.

Problem 4: Student result prediction

Aim

To implement FIND-S as a Python class with fit and predict methods and to use the learned hypothesis to classify new student samples.

Algorithm

1. Initialise h to the most specific hypothesis in H, <0, 0, ..., 0>.
2. For each training example x:

   If x is a positive example: for each attribute constraint a in h, if a is satisfied by x do nothing; otherwise replace a in h by the next more general constraint that is satisfied by x (the first positive example fills in its own values and a conflicting value becomes ?).

   If x is a negative example: ignore it and leave h unchanged.
3. Output the hypothesis h.

Program:

find_s_class.py — Python 3

```python
# Program 1(d): FIND-S implemented as a Python class with prediction
class FindS:
    def __init__(self, n):
        self.h = ['0'] * n

    def fit(self, X, y, positive):
        for x, label in zip(X, y):
            if label != positive:
                continue
            for i, v in enumerate(x):
                if self.h[i] == '0':
                    self.h[i] = v
                elif self.h[i] != v:
                    self.h[i] = '?'
        return self

```

find_s_class.py — Python 3

```python
    def predict(self, x):
        return all(a == '?' or a == b for a, b in zip(self.h, x))

X = [("High", "High",   "Done",    "Good"),
     ("High", "Medium", "Done",    "Good"),
     ("Low",  "Low",    "Pending", "Poor"),
     ("High", "High",   "Done",    "Average"),
     ("Low",  "Medium", "Pending", "Average")]
y = ["Pass", "Pass", "Fail", "Pass", "Fail"]

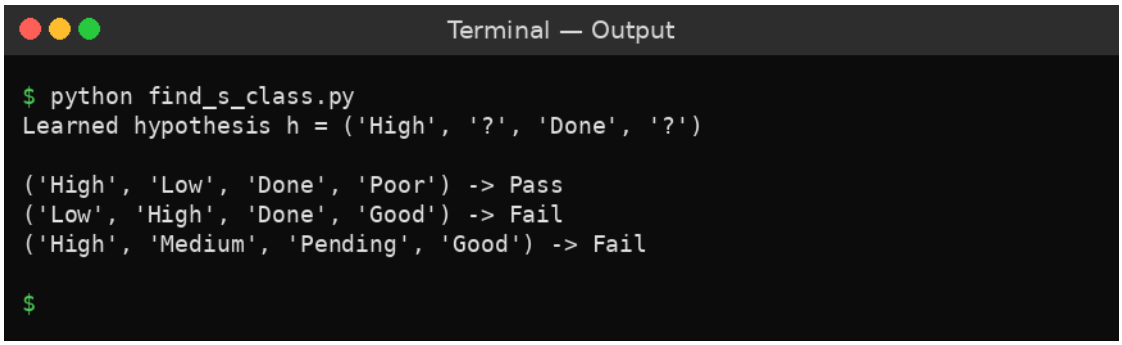
model = FindS(4).fit(X, y, "Pass")
print("Learned hypothesis h =", tuple(model.h))
print()
for t in [("High", "Low", "Done", "Poor"), ("Low", "High", "Done", "Good"),
          ("High", "Medium", "Pending", "Good")]:
    print(t, "->", "Pass" if model.predict(t) else "Fail")
```

Input

Student result data set: 5 examples, 4 attributes (Attendance, StudyHours, Assignments, Internal) and the target concept Result (Pass / Fail). The data is stored in the program as lists.

Attendance	StudyHours	Assignments	Internal	Result
High	High	Done	Good	Pass
High	Medium	Done	Good	Pass
Low	Low	Pending	Poor	Fail
High	High	Done	Average	Pass
Low	Medium	Pending	Average	Fail

Output



Result

The learned hypothesis is h = <High, ?, Done, ?>. Using it, (High, Low, Done, Poor) is classified Pass, while (Low, High, Done, Good) and (High, Medium, Pending, Good) are classified Fail because each violates one of the two remaining constraints. A student is predicted to pass only when attendance is High and the assignments are Done.

Problem 5: Animal classification

Aim

To demonstrate that FIND-S ignores negative examples by running it on all the examples and on the positive examples only, and to verify that the final hypothesis is consistent with the training data.

Algorithm

1. Initialise h to the first positive training example.
2. For each remaining training example x:

   If x is a positive example: for each attribute constraint a in h, if a is satisfied by x do nothing; otherwise replace a in h by the more general constraint ? (the value is no longer fixed).

   If x is a negative example: ignore it and leave h unchanged.
3. Output the hypothesis h.

Program:

find_s_negatives.py — Python 3

```python
# Program 1(e): FIND-S - negative examples are ignored; consistency check
def generalise(h, x):
    return [a if a == b else '?' for a, b in zip(h, x)]

def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))

def find_s(examples):
    h = None
    for x, label in examples:
        if label == "Yes":
            h = list(x) if h is None else generalise(h, x)
    return h

```

find_s_negatives.py — Python 3

```python
train = [
    (("Fur",      "Live", "Four", "Land"),  "Yes"),
    (("Fur",      "Live", "Two",  "Air"),   "Yes"),
    (("Scales",   "Eggs", "Four", "Land"),  "No"),
    (("Smooth",   "Live", "None", "Water"), "Yes"),
    (("Feathers", "Eggs", "Two",  "Air"),   "No"),
]
positives = [e for e in train if e[1] == "Yes"]

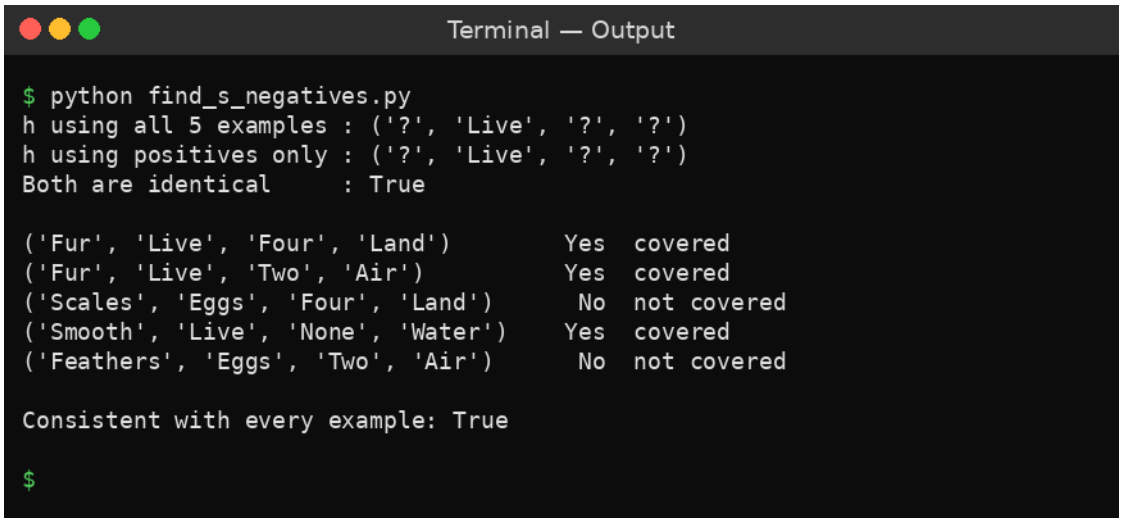
h_all = find_s(train)
h_pos = find_s(positives)
print("h using all 5 examples :", tuple(h_all))
print("h using positives only :", tuple(h_pos))
print("Both are identical     :", h_all == h_pos)
print()
for x, label in train:
    status = "covered" if covers(h_all, x) else "not covered"
    print(f"{str(x):<38} {label:>3}  {status}")
ok = all(covers(h_all, x) == (y == "Yes") for x, y in train)
print()
print("Consistent with every example:", ok)
```

Input

Animal data set: 5 examples, 4 attributes (Covering, Birth, Legs, Habitat) and the target concept Mammal (Yes / No). The data is stored in the program as tuples.

Covering	Birth	Legs	Habitat	Mammal
Fur	Live	Four	Land	Yes
Fur	Live	Two	Air	Yes
Scales	Eggs	Four	Land	No
Smooth	Live	None	Water	Yes
Feathers	Eggs	Two	Air	No

Output



Result

Both runs give the same hypothesis, <?, Live, ?, ?>, which shows that the negative examples have no effect on FIND-S. The hypothesis covers all three positive examples and rejects both negative examples, so it is consistent with the whole training set. Live birth is the only attribute value shared by every mammal in the data.